Click to register for Certification exam

If already registered, click to check your payment status

Course outline

About NPTEL

How does an NPTEL online course work?

Week 1

Week 2

Week 3

week 4

Week 5

Week 6

Week 7

- ○ Snakes and Ladders - Not on the Board
- ○ Snakes and Ladders - Not on the Board - Part 01
- ○ Snakes and Ladders - Not on the Board - Part 02
- ○ Snakes and Ladders - Not on the Board - Part 03
- ○ Snakes and Ladders - Not on the Board - Part 04
- ○ Snakes and Ladders - Not on the Board - Part 05
- ○ Snakes and Ladders - Not on the Board - Part 06
- ○ Spiral Traversing - Let's Animate
- ○ Spiral Traversing - Let's Animate - Part 01
- ○ Spiral Traversing - Let's Animate - Part 02
- ○ Spiral Traversing - Let's Animate - Part 03
- ○ Spiral Traversing - Let's Animate - Part 04
- ○ Spiral Traversing - Let's Animate - Part 05
- ○ Spiral Traversing - Let's Animate - Part 06
- ○ Spiral Traversing - Let's Animate - Part 07
- ○ GPS - Track the route
- ○ GPS - Track the route - Part 01
- ○ GPS - Track the route - Part 02
- ○ GPS - Track the route - Part 03
- ○ GPS - Track the route - Part 04
- ○ Week 7 Feedback Form: The Joy of Computing using Python
- ○ **Quiz: Week 7 : Assignment 7**
- ○ Week 7 : Programming Assignment 1
- ○ Week 7 : Programming Assignment 2
- ○ Week 7 : Programming Assignment 3

Text Transcripts

Download Videos

Week 7 : Assignment 7

Assignment not submitted **Due date: 2026-03-11, 23:59 IST.**

Case Study: Arjun and the Archivist's Rules

Arjun was an apprentice archivist in the **City of Timelines**, a place where knowledge was preserved under strict rules. Every object stored in the archive followed a **data discipline**; nothing was random, and nothing was accidental.

One afternoon, the Chief Archivist handed Arjun two enchanted records.

"These are not just containers," she said.
"They decide **how information behaves**."

Record One: The Chronicle Scroll

The first record was called the **Chronicle Scroll**.

"This scroll preserves history **exactly as it is written**," said the Archivist.

"Once sealed, its contents can never be altered."

The Archivist explained that the Chronicle Scroll works like a **Tuple**.

Chronicle Scroll = Tuple

Arjun recorded the sequence of royal events:

chronicle = ("Coronation", "War Declared", "Peace Treaty")

He reviewed the entries:

print(chronicle[1])

Output:

War Declared

Encouraged, Arjun tried to revise history:

chronicle[1] = "Secret Negotiation"

The archive rejected the change immediately.

What Arjun Observed

- Events stayed in the **same order**
- Each event had a **fixed position**
- Once recorded, entries **could not be changed**
- The scroll allowed **repeated events** if history demanded it

Inference:
Tuples preserve order, are immutable, and can store duplicates.
They use **round brackets ()**.

Record Two: The Artifact Vault

The second record was called the **Artifact Vault**.

"This vault only cares about **what exists**, not **how it is arranged**," said the Archivist.

"And it never keeps the same artifact twice."

The vault behaves like a **Set**.

Artifact Vault = Set

Arjun logged collected artifacts:

artifacts = {"ring", "amulet", "ring", "scroll"}

When he checked the vault, he saw:

print(artifacts)

Output (order may vary):

{'ring', 'amulet', 'scroll'}

The duplicate artifact had vanished.

Managing the Artifact Vault

Unlike the Chronicle Scroll, the vault was flexible.

Arjun added a new artifact:

artifacts.add("crystal")

Later, he removed one:

artifacts.remove("ring")

The vault updated instantly.

What Arjun Observed

- Order was **unpredictable**
- Duplicate artifacts were **automatically ignored**
- Items could be **added or removed freely**
- The vault focused only on **uniqueness**

Inference:
Sets are unordered, mutable, and store only unique elements.
They use **curly brackets {}**.

1) Arjun attempted to change an entry inside the Chronicle Scroll using: *1 point*
chronicle[1] = "Secret Negotiation"
Why did this action fail?

○ Because tuples do not support indexing
○ Because the element did not exist
○ Because the Chronicle Scroll preserves order only
○ Because the Chronicle Scroll is immutable

2) When Arjun created the Artifact Vault as: *1 point*
artifacts = {"ring", "amulet", "ring", "scroll"}
Why did the final vault contain fewer items than expected?

○ Because sets automatically sort items
○ Because duplicate elements are ignored
○ Because only strings are allowed
○ Because indexing is not supported

3) Which operation was possible on the Artifact Vault but impossible on the Chronicle Scroll? *1 point*

○ Accessing an element using index
○ Preserving insertion order
○ Modifying the container after creation
○ Allowing duplicate values

4) Arjun noticed that printing the Artifact Vault showed items in an unpredictable order. What caused this behavior? *1 point*

○ The vault was empty initially
○ Python randomly shuffles set elements
○ Sets do not maintain insertion order
○ Duplicate elements change ordering

5) Which of the following would raise an error if attempted on the Chronicle Scroll? *1 point*

○ Checking length
○ Accessing an element
○ Iterating through elements
○ Updating an existing element

6) Why was the Artifact Vault more suitable for tracking collected artifacts? *1 point*

○ It supports indexing
○ It allows duplicates
○ It guarantees uniqueness
○ It preserves historical order

7) If Arjun needed to ensure that historical records could never be altered, which structure was most appropriate? *1 point*

○ List
○ Set
○ Dictionary
○ Tuple

8) Which statement about indexing is TRUE based on the case study? *1 point*

○ Both tuples and sets support indexing
○ Only sets support indexing
○ Only tuples support indexing
○ Neither supports indexing

9) Arjun removed an item from the Artifact Vault successfully. Why was this operation safe? *1 point*

○ Because sets allow modification
○ Because tuples allow removal
○ Because removal works only on strings
○ Because indexing was used

10)Which inference correctly matches both records used by Arjun? *1 point*

○ Chronicle Scroll → unordered, mutable
○ Artifact Vault → ordered, immutable
○ Chronicle Scroll → ordered, immutable
○ Artifact Vault → ordered, duplicate-allowed

Meera and the Community Learning Center

Meera volunteered at a **community learning center** that offered short-term courses such as Python Basics, Data Analysis, and Web Development.

The center faced a recurring problem every month:

- Student registrations came from different sources
- Course details were fixed once announced
- Attendance had to be tracked per student
- Records needed to be stored permanently for audits

Meera was asked to build a **simple data system in Python** that could manage all this information reliably.

Step 1: Fixed Course Information (Tuples)

At the start of the month, course details were finalized and **never changed afterward**.

Each course had:

- Course ID
- Course Name
- Duration (weeks)

Meera stored this data as **tuples.**

```
python_course = (101, "Python Basics", 6)
data_course = (102, "Data Analysis", 8)
web_course = (103, "Web Development", 10)
```

Step 2: Managing Student Registrations (Lists)

Student registrations arrived **daily**, and new names had to be added continuously.

Meera maintained a list of registered students.

```
students = ["Aarav", "Neha", "Ishaan"]
students.append("Mehul")
students.append("Neha")
```

Step 3: Mapping Students to Courses (Dictionary)

Each student enrolled in **one course only**, and Meera needed a clear mapping.

She used a **dictionary** where:

- Key → Student name
- Value → Course tuple

```
enrollments = {
    "Aarav": python_course,
    "Neha": data_course,
    "Ishaan": python_course,
    "Mehul": web_course
}
```

Step 4: Tracking Attendance per Student (Dictionary + List)

Attendance changed every day, so Meera needed a **mutable structure**.

She decided:

- Each student would have a list of attended session numbers

```
attendance = {
    "Aarav": [1, 2, 3],
    "Neha": [1, 2],
    "Ishaan": [1, 3],
    "Mehul": []
}
```

Later updates:

```
attendance["Mehul"].append(1)
attendance["Neha"].append(3)
```

Step 5: Saving Records to a File (File Handling)

At the end of the week, Meera needed to **store enrollment data permanently**.

She wrote student-course mappings to a file.

```
with open("enrollments.txt", "w") as file:
    for student, course in enrollments.items():
        file.write(f"{student} - {course[1]} ({course[2]} weeks)\n")
```

Step 6: Reading Stored Records Later

During an audit, Meera retrieved the saved data.

```
with open("enrollments.txt", "r") as file:
    records = file.readlines()

for line in records:
    print(line.strip())
```

11)Suppose the following change is made **before writing to the file**: *1 point*

```
python_course = (101, "Python Basics", 6)
python_course[2] = 7
```

What would be the **first issue** encountered in the program?

○ The file will store incorrect duration
○ A runtime error will occur due to tuple immutability
○ The dictionary enrollments will update automatically
○ The list students will change size

12)Consider this code is added **after reading from the file**: *1 point*

```
records = file.readlines()
records.append("Rohan - Python Basics (6 weeks)\n")
```

Which statement is TRUE?

○ A file write error occurs
○ The file content changes automatically
○ The new record is saved permanently in enrollments.txt
○ Only the in-memory list is modified

13)Assume this code is executed: *1 point*

```
attendance["Aarav"].append(4)
enrollments["Aarav"] = (101, "Python Basics", 6)
```

Which statement best explains what happens?

○ Both operations fail
○ Both operations succeed
○ Attendance succeeds, enrollment fails
○ Enrollment succeeds, attendance fails

14)If Meera replaces this line: *1 point*

```
students = ["Aarav", "Neha", "Ishaan"]
```

with:

```
students = ("Aarav", "Neha", "Ishaan")
students.append("Mehul")
```

What is the correct outcome?

○ The student is added successfully
○ A runtime error occurs due to unsupported operation
○ The tuple expands automatically
○ The enrollment file updates

15)Which design choice in the case study **most directly protects course data from accidental modification**? *1 point*

○ Using formatted strings while writing files
○ Using lists for attendance tracking
○ Reading files using readlines()
○ Storing course details as tuples inside dictionaries

You may submit any number of times before the due date. The final submission will be considered for grading.

Submit Answers